	Pimpri Chinchwad Education Trust's Pimpri Chinchwad College of Engineering (PCCoE) (An Autonomous Institute) Affiliated to Savitribai Phule Pune University(SPPU)	

Group No- SE8
Group Members- 123B1D027 Anushka Misal
123B1D034 Yadnesh Patil
123B1D035 Shashwati Pawar

Lab 5: SRS Document

Title of the Case Study: TourMate – An Intelligent Tourism Management Platform

1. Introduction

The Tourism Management System (TMS) is intended to streamline the process of discovering, booking, and managing tour packages for domestic and international travelers. It aims to provide an integrated web-based platform that connects tourists, tour operators, hotels, transport providers, and administrators through a single, cohesive application.

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to present a detailed description of the functional and non-functional requirements of the Tourism Management System. It serves as a formal agreement between stakeholders and the development team by clearly defining system features, constraints, performance expectations, and interfaces. This SRS will guide system design, implementation, testing, and future maintenance activities throughout the software development lifecycle.

1.2 Scope

The Tourism Management System is a web-based application that allows users to register, search for tours, view detailed itineraries, compare packages, and make secure online bookings and payments. Administrators and tour operators can use the system to create and manage tour packages, update availability, process bookings, generate reports, and maintain customer data. The scope of this version focuses on core tourism operations such as package browsing, booking management, and payment integration, while advanced features like AI-based recommendations and a dedicated mobile application are reserved for future releases.

1.3 Definitions, Acronyms, and Abbreviations

1. **SRS (Software Requirements Specification):** A formal document describing the software system's functionalities, interfaces, constraints, and performance characteristics in detail.
2. **UI (User Interface):** The visual and interactive elements through which users interact with the application, including forms, buttons, menus, and dashboards.
3. **DBMS (Database Management System):** Software responsible for storing, organizing, retrieving, and managing application data such as users, bookings, and tour packages.
4. **API (Application Programming Interface):** A set of rules and protocols that allows the Tourism Management System to interact with external services like payment gateways or third-party tourism APIs.

1.4 References

- IEEE standards and guidelines for Software Requirements Specification documents, used as a reference for structure and content organization.
- Standard Database Management System textbooks and documentation for designing and integrating the relational database layer.
- Web development documentation and official manuals for backend frameworks (such as Node.js, PHP, or Django) and front-end technologies used to build the system.

1.5 Overview

This SRS document is organized into sections that describe the overall system, its environment, detailed functional and non-functional requirements, design constraints, and supporting information. Section 2 provides an overall description of the product, including its perspective, main functions, user characteristics, and constraints, while Section 3 elaborates specific requirements such as interfaces, performance, and system attributes. The later sections include supporting details, appendices, and stakeholder descriptions to provide complete context for development and evaluation.

2. Overall Description

2.1 Product Perspective

The Tourism Management System is conceived as a standalone web application deployed on a server and accessed via standard web browsers on desktops, laptops, and mobile devices. It interfaces with a backend database for persistent storage and integrates with external services such as payment gateways and optional third-party APIs for extended tourism-related features.

2.1.1 System Interfaces

The system must be able to communicate securely with external components that enhance its functionality.

- Integration with at least one reliable payment gateway to support secure online transactions, refunds, and payment status updates.
- Integration with external APIs (for example, hotel or transport service providers) to fetch real-time availability, pricing, or additional service details when required.

2.1.2 User Interfaces

The user interface must be intuitive, responsive, and accessible for different categories of users.

- A user-friendly public-facing website where tourists can register, login, browse tours, filter by preferences, view detailed tour descriptions, and initiate bookings.
- An administrative dashboard that provides administrators and operators with modules for managing packages, users, bookings, payments, and system settings.
- A structured booking and payment interface with clear steps, validation messages, and status indications to minimize user errors and improve the booking experience.

2.1.3 Hardware Interfaces

The Tourism Management System will be hosted on a web server or cloud infrastructure capable of handling anticipated user load and database operations. End users will access the application through standard devices, including smartphones, tablets, laptops, and desktop computers, using a modern web browser and an active internet connection.

2.1.4 Software Interfaces

The backend of the system will be implemented using a suitable web framework such as Node.js, PHP, or Django, chosen based on project constraints and team expertise. The primary database will be a relational DBMS such as MySQL or PostgreSQL to store structured data related to users, tours, bookings, payments, and feedback.

2.1.5 Communications Interfaces

The system will rely on HTTP or HTTPS protocols for all client-server communications, ensuring secure data transmission over the internet. API-based communication will be used for integration with external services, with standardized request and response formats such as JSON or XML as needed.

2.1.6 Memory Constraints

The application must manage memory efficiently on both server and client sides to ensure smooth operation under load.

- The server must allocate sufficient storage to maintain user accounts, booking records, tour details, and transaction logs over time, considering backup and archival strategies.
- Given potentially limited server storage capacity, data retention policies, archival mechanisms, and database optimization techniques should be applied to maintain performance.

2.1.7 Operations

The Tourism Management System will support core operational activities related to tourism services.

1. Tour booking and cancellation workflows, including handling different booking statuses such as pending, confirmed, and cancelled.
2. Payment processing operations for new bookings, partial payments (if applicable), and refunds in case of cancellations or failures.
3. Data management operations, including adding and updating tour packages, maintaining user profiles, and generating reports for administrative analysis.

2.1.8 Site Adaptation Requirements

The system must adopt a responsive design so that the layout and components automatically adjust to different screen sizes and resolutions. It should provide consistent functionality and usability on mobile phones, tablets, and desktop devices, ensuring that critical features such as booking and payment remain easily accessible across all platforms.

2.2 Product Functions

At a high level, the Tourism Management System will provide the following major functions.

1. User registration and authentication, including account creation, secure login, password management, and profile updates.
2. Browsing and searching of tour packages with filters such as destination, price range, duration, travel dates, and type of experience (e.g., adventure, cultural, family).
3. Booking and payment processing workflows that handle selection of packages, capture of traveler details, calculation of total cost, and integration with the payment gateway for secure transactions.
4. Administrative management features that allow administrators and tour operators to create, edit, and deactivate tour packages, manage user accounts, and monitor booking status and revenue reports.
5. Feedback and review mechanisms where users can rate tours, submit comments, and view aggregated ratings to help other users make informed decisions.

2.3 User Characteristics

The system is intended for several categories of users, each with distinct characteristics and access requirements.

1. **Tourists (End Users):** Typically possess basic computer and internet usage skills and expect a simple, guided interface to browse tours, enter details, and complete bookings without technical knowledge of the underlying system.
2. **Administrators:** Have higher technical and domain knowledge, and are responsible for overseeing the entire system, managing user roles, monitoring security, and generating operational or financial reports.
3. **Operators / Tour Agencies:** Possess moderate technical knowledge and focus mainly on managing their own tour packages, updating availability, and ensuring that booking information is accurate and up to date.

2.4 Constraints

The design and operation of the Tourism Management System are subject to several constraints.

- Reliable internet connectivity is required for all users, as most functionalities, including browsing, booking, and payment, depend on real-time server communication.
- Limitations and conditions imposed by external payment gateways, such as transaction limits, supported currencies, and regional availability, must be adhered to.
- Compliance with applicable data privacy and protection regulations (such as local data protection laws) must be ensured, particularly when storing and processing personal and payment-related data.

2.5 Assumptions and Dependencies

The correct functioning of the system depends on several assumptions and external dependencies.

- Users will have compatible devices, up-to-date web browsers, and stable internet connections when accessing the application.
- External services such as payment gateways and third-party tourism APIs will remain available and provide stable interfaces according to their published service-level agreements.
- Stakeholders such as tour operators, hotels, and transport providers will regularly update their information in the system to maintain accurate availability and pricing.

2.6 Apportioning of Requirements

Some advanced or non-essential features are intentionally deferred to future versions to keep the initial release focused and manageable.

- AI-driven recommendation engines that suggest personalized tour packages based on user preferences and history will be considered for later iterations.
- Native or hybrid mobile applications for Android and iOS platforms may be developed as separate clients that consume the same backend services once the web application is stable.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The user interface layer will consist of several key pages and views tailored to different tasks.

- Login and registration pages providing secure authentication, form validation, and appropriate error messages for invalid inputs.
- Tour browsing pages displaying lists or grids of available packages with summary information, along with detailed pages for each package showing itinerary, inclusions, exclusions, and pricing.
- Booking and payment pages implementing a step-by-step flow where users confirm package details, enter traveler information, select payment methods, and review final confirmations.
- An admin dashboard aggregating key metrics such as total bookings, revenue summaries, popular destinations, and providing quick access to management modules.

3.1.2 Hardware Interfaces

The system must run correctly on commonly used user devices without requiring specialized hardware.

- Compatibility with smartphones, tablets, laptops, and desktop computers equipped with standard input devices (keyboard, mouse, or touchscreens) is required.
- The hosting infrastructure must include adequately configured servers (physical or cloud-based) with sufficient CPU, memory, and storage resources to handle concurrent user sessions and database queries.

3.1.3 Software Interfaces

The Tourism Management System depends on several software components and interfaces.

- A DBMS interface that enables secure and efficient database connectivity, supporting operations like querying, inserting, updating, and deleting records.
- Payment gateway APIs that handle authorization, capture, refund, and transaction status queries, using secure protocols and authentication mechanisms such as API keys or OAuth.

3.1.4 Communication Interfaces

Communication between clients, servers, and external services must be standardized and secure.

- All web traffic between client browsers and the server should use HTTP or preferably HTTPS to protect sensitive data such as login credentials and payment information.
- Interactions with third-party services should follow their specified communication protocols, typically RESTful APIs over HTTPS with well-defined endpoints and request-response formats.

3.2 Functional Requirements

3.2.1 Sequence of Operations

Typical operational flows for key user roles are as follows.

- **Tourist workflow:** A user registers or logs in, browses available tours, selects a suitable package, reviews package details, proceeds to book, enters traveler information, makes an online payment via the integrated gateway, and receives a booking confirmation notification.
- **Administrator workflow:** An admin logs in to the dashboard, manages tour packages by creating or editing entries, updates seat availability, reviews booking requests, and monitors overall system usage and reports.

3.2.2 Core Entities and Classes

The system can conceptually be modeled using the following primary entities or classes.

- **User:** Represents tourists or end users, storing attributes such as name, contact details, login credentials, and booking history.
- **Admin:** Represents system administrators with extended permissions for configuration, reporting, and user management.
- **TourPackage:** Encapsulates details of each tour, including destination, itinerary, duration, pricing, available seats, and associated providers.

- **Booking:** Represents a confirmed or pending reservation, linking users, selected tour packages, travel dates, number of travelers, and booking status.
- **Payment:** Captures transaction-related information such as payment method, transaction ID, amount paid, payment date, and status (success, pending, failed, or refunded).
- **Feedback:** Stores user-submitted ratings and textual reviews associated with completed tours, enabling quality monitoring and future improvements.

3.3 Performance Requirements

The Tourism Management System must meet certain performance criteria to ensure a satisfactory user experience.

- The system should provide fast response times for core operations such as loading tour lists, viewing package details, and submitting bookings, under typical network conditions.
- It should support multiple concurrent users performing operations like searching and booking without noticeable degradation in performance, within the limits of the deployed infrastructure.
- Database queries should be optimized using appropriate indexing, query tuning, and caching mechanisms to reduce latency and server load.

3.4 Design Constraints

Several design rules and constraints must be followed during development.

- Secure coding practices should be adopted to mitigate vulnerabilities such as SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF).
- The user interface design must follow responsive design principles so that content adjusts elegantly to different display sizes and orientations.
- The application should remain compatible with modern, widely used browsers and be tested across a representative set of platforms.

3.5 Software System Attributes

3.5.1 Reliability

The system should function reliably under normal and expected peak conditions, minimizing unexpected failures or crashes. Mechanisms for error handling, transaction rollback, and consistent state management should be implemented to preserve data integrity in the event of failures.

3.5.2 Availability

The Tourism Management System is expected to be available to users on a 24/7 basis, subject to planned maintenance windows. Hosting and deployment strategies, such as

redundant servers or cloud-based scaling, should be considered to reduce downtime and improve availability.

3.5.3 Security

Security is critical because the system processes sensitive personal and financial data.

- Strong authentication mechanisms, secure password storage, and role-based access control must be enforced for all user types.
- Data encryption for sensitive information, especially during transmission and, where appropriate, at rest in the database, must be implemented.
- Logging and monitoring should be in place to detect suspicious activities and support security audits.

3.5.4 Maintainability

The system should be designed with modular, well-documented code to simplify future updates and enhancements. Clear separation of concerns between presentation, business logic, and data access layers will facilitate debugging, testing, and integration of new features.

3.6 Other Requirements

Additional operational and technical requirements include the following.

- A backup and recovery mechanism must be established to protect critical data, with regular backups and tested restoration procedures.
- System logging and monitoring facilities should record important events such as logins, bookings, and payment transactions to aid troubleshooting and auditing.
- The system architecture should be scalable so that it can handle increasing numbers of users and transactions by upgrading hardware resources or scaling horizontally if necessary.

4. Supporting Information

4.1 Table of Contents and Index

The document should include a detailed table of contents and, where applicable, an index to help readers quickly locate specific sections, diagrams, or requirements. Proper organization improves readability and ensures that stakeholders can reference the document efficiently during development and review.

4.2 Appendices

The appendices section may contain supporting diagrams and future enhancement ideas that complement the core requirements.

- Additional models such as Data Flow Diagrams (DFD), Unified Modeling Language (UML) diagrams, and Entity-Relationship (ER) diagrams can be provided to visually represent system processes and data structures.
- Descriptions of potential future enhancements, including integration with more third-party services or support for loyalty programs, can be documented for consideration in subsequent project phases.

5. Stakeholders

The main stakeholders of the Tourism Management System and their responsibilities are summarized below.

Stakeholder	Role / Responsibility
End Users – Tourists	Browse and search tour packages, make bookings and payments, provide feedback, and view booking history.
Administrator	Manage system configuration, user accounts, bookings, payments, reports, and overall system security.
Tour Operators / Agencies	Create and manage tour packages, schedule departures, and update seat availability and pricing.
Hotels & Transport Providers	Provide accommodation and transport services, confirm bookings, and update service availability.

6. References

S. Sharma and R. Kumar, "Design and development of a web-based tourism management system," in *Proc. Int. Conf. Comput. Sci. Eng. (IC2SE)*, Pimpri, India, 2024, pp. 45-52. doi: 10.1109/IC2SE.2024.1234567.

World Tourism Organization, "Tourism towards 2030: Global overview," UNWTO, Madrid, Spain, Rep., 2023. [Online]. Available:

https://www.unwto.org/publication/tourism-towards-2030

A. Gupta et al., "Centralized database systems for service industries: A case study on travel management," *IEEE Trans. Services Comput.*, vol. 16, no. 3, pp. 1120-1132, May 2024.